LAB-9 : File Handing in C

Objective

- To be familiar with file handling, types of file and reading and writing in files using C language.

PROGRAM #1 : Program in C to read content from 'tale-txt' and display all the content.

PROGRAM ANALYSIS

To develop a program in C to read content from 'tale.txt' and display all the content, we file pointer to handle file operations also a character to store each character from the file.

ALGORITHM:

Step 1 : START

Step 2 : Declare file pointer fp and characater variable ch

Step 3 : Open the file "tale.txt" in read mode using fopen( )

Step 4 : If fp = NULL
- Print Error !
- Terminate the program

Step 5 : do
Read a character using ch = fgetc (fp)
Print the character

Step 6 : If ch = EOF

Step 7 : Close the file using fclose (fp)

Step 8 : Print "Program Completed"

Step 9 : STOP

C-PROGRAM:

```
/* Program in C to read content from 'fale.txt' and
display the content */
#include <stdio.h>
int main ( )
{
 FILE *fp;
 Char ch;
 fp = fopen ("false.txt", 'r');
 if (fp == NULL)
 {
 printf ("File error!\n");
 exit(1);
 }
 do
  {
   ch = fgetc(fp);
 printf ("%c", ch);
  }while (ch != EOF);
 fclose (fp);
 printf ("\n\n Program Completed \n");
 return 0;
 }
```

PROGRM #2: Program in C to count number of vowels "tile.txt" and display the count.

PROGRAM ANALYSIS

To develop a program to count number of vowels in "fale.txt" and display the count, we open the file first in read mode. The we use fget c) to read character one by one and check the condition i.e. if each character is vowel increasing the counter and finally display it.

ALGORITHM:

Step 1: Start

Step 2: Declare file pointer fp, character variable ch and integer variable count which is initialized at 0.

Step 3: Open the file "fale.txt" in read mode.

~~Step 4: Open the file "fale.txt"~~

Step 4: Check if fp = NULL
PRINT Error!
Terminate the program
Apply condition

Step 5: Apply condition while ((ch = fgetc(fp)) != EOF)
{
if (ch = 'a' || ch = = 'e' || ch = = 'i' || ch = = 'o' || ch = = 'u' ||
ch = 'A' || ch = = 'E' || ch = 'I' || ch = = "O" || ch = = 'U')
count ++

Step 6: Close the file using fclose (fp)

Step 7: Print number of vowels

Step 8: Stop

C-PROGRAM

```
// program to count number of vowels in "fale.txt"
    # include <stdio.h>
    # include <# stdlib.h>
    int main ( )
    {
      FILE *fp;
      Char ch;
      int count = 0;
      fp = fopen ("fale.txt", "r");
      if (fp = NULL)
      {
      printf ("file Error!\n");
      exit (1);
      }
      while ((ch = fgetc (fp)) != EOF)
      {
      if (ch = = 'a' || ch = = 'e' || ch = = 'i' || ch = 'o' || ch = = 'u' ||
```

```
Ch=='A'||ch==='E'||ch==='I'||ch==='O'||ch==='U')
{
count ++;
}
fclose (fp);
printf("Number of vowels = %d\n", count);
return 0;
}
```

PROGRAM #3: Write a program in C to illustrate concept of fseek( ) and ftell( ).

PROBLEM ANALYSIS

The program demonstrates file pointer movement using fseek( ) and position tracking using ftell( ). It shows how to move to specific locations in a file and determine the current.

ALGORITHM:

Step 1: START
Step 2: Open a file in read mode
Step 3: use fseek( ) to move file point to desired position
Step 4: Use ftell( ) to get current position
Step 5: Display position
Step 6: Repeat steps for different positions (beginning, current, end)
Step 7: Close the file
Step 8: STOP

C-PROGRAM

```
#include <stdio.h>
int main( )
{
FILE * fp;
fp=fopen("file.txt","r");
if (fp==NULL)
{
printf("file not found!\n");
return 1;
}
fseek(fp,0,SEEK_END);
printf("Position at end: %d\n", ftell(fp));
fseek(fp,0,SEEK_SET);
```

```
printf("Position at beginning: %d\n", ftell(fp));
fseek(fp, 10, SEEK_SET);
printf("Position after 10 bytes: %d\n", ftell(fp));
fclose(fp);
return 0;
}
```

OUTPUT:

Example: 50 bytes (file size)
Position at end: 50
Position of begining: 0
Position after 10 bytes: 10

RESULT:
Ouput was successfully printed.

CONCLUSION:

In conclusion, this file handling, types of file and reading and writing to files helped us to understand the basic concept of C. We learned how to input & output, handled diff. problems.